

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze
8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I M.SHARMILA(192572139) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF

1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF

1. 1NF

```
mysql> select*from university_data;
```

student_id	student_name	course_id	course_name	instructor	instructor_email	department
S101	Arun	C101	DBMS	Dr. Kumar	kumar@univ.com	CSE
S101	Arun	C102	OS	Dr. Ravi	ravi@univ.com	CSE
S102	Priya	C101	DBMS	Dr. Kumar	kumar@univ.com	CSE
S103	Kiran	C103	AI	Dr. Meena	meena@univ.com	AI

```
4 rows in set (0.00 sec)
```

2NF

```
mysql> SELECT * FROM student;
```

student_id	student_name
S101	Arun
S102	Priya
S103	Kiran

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM course;
```

course_id	course_name	instructor	instructor_email	department
C101	DBMS	Dr. Kumar	kumar@univ.com	CSE
C102	OS	Dr. Ravi	ravi@univ.com	CSE
C103	AI	Dr. Meena	meena@univ.com	AI

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM enrollment;
```

student_id	course_id
S101	C101
S102	C101
S101	C102
S103	C103

```
4 rows in set (0.00 sec)
```

3NF

```
-- Final Result: Join to show complete information
mysql> SELECT
-> s.student_id,
-> s.student_name,
-> c.course_id,
-> c.course_name,
-> i.instructor_name,
-> i.instructor_email,
-> i.department
-> FROM student s
-> JOIN enrollment e ON s.student_id = e.student_id
-> JOIN course c ON e.course_id = c.course_id
-> JOIN instructor i ON e.instructor_id = i.instructor_id;
+-----+-----+-----+-----+-----+-----+
| student_id | student_name | course_id | course_name | instructor_name | instructor_email | department |
+-----+-----+-----+-----+-----+-----+
| 5181       | Arun         | C181      | DBMS        | Dr. Kumar       | kumar@univ.com   | CSE        |
| 5182       | Arun         | C182      | OS          | Dr. Ravi        | ravi@univ.com    | CSE        |
| 5183       | Priya        | C181      | DBMS        | Dr. Kumar       | kumar@univ.com   | CSE        |
| 5183       | Kiran        | C183      | AI          | Dr. Heena       | heena@univ.com   | AI         |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

2.A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach

```
mysql> SELECT e.Emp_ID, e.Emp_Name, COUNT(ed.Dept_ID) AS Department_Count
-> FROM Employee e
-> JOIN Employee_Department ed ON e.Emp_ID = ed.Emp_ID
-> GROUP BY e.Emp_ID, e.Emp_Name
-> HAVING COUNT(ed.Dept_ID) > 1;

+-----+-----+-----+
| Emp_ID | Emp_Name | Department_Count |
+-----+-----+-----+
| 101    | Arun     | 2                |
| 103    | Ravi     | 2                |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

QUERIES EXPLANATION

Query:

SQL

```
SELECT e.Emp_ID, e.Emp_Name, COUNT(ed.Dept_ID) AS Department_Count
FROM Employee e
JOIN Employee_Department ed
ON e.Emp_ID = ed.Emp_ID
GROUP BY e.Emp_ID, e.Emp_Name
HAVING COUNT(ed.Dept_ID) > 1;
```

Simple Explanation

1. SELECT e.Emp_ID, e.Emp_Name
→ Selects the employee ID and employee name.

2. COUNT(ed.Dept_ID)
→ Counts how many departments each employee works in.
3. FROM Employee e
→ Takes employee details from the Employee table.
e is an alias for Employee.
4. JOIN Employee_Department ed
→ Combines the employee table with the department assignment table.
5. ON e.Emp_ID = ed.Emp_ID
→ Matches the same employee in both tables.
6. GROUP BY e.Emp_ID, e.Emp_Name
→ Groups all department records belonging to each employee.
7. HAVING COUNT(ed.Dept_ID) > 1
→ Displays only employees who belong to **more than one department**.

Result

Emp_ID	Emp_Name	Department_Count
101	Arun	2
103	Ravi	2

Conclusion: The query finds employees who are working in **multiple departments** by joining the tables, counting departments, and filtering counts greater than 1.

3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+
| Emp_ID | Emp_Name |
+-----+-----+
| 101    | Arun     |
| 102    | Priya    |
| 103    | Ravi     |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM DEPARTMENT;
+-----+-----+
| Dept_ID | Dept_Name |
+-----+-----+
| 10      | HR        |
| 20      | Sales     |
| 30      | IT        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM EMP_DEPT;
+-----+-----+
| Emp_ID | Dept_ID |
+-----+-----+
| 101    | 10      |
```

QUERIES EXPLANATION

Query 1: Create Employee Table

SQL

```
CREATE TABLE EMPLOYEE (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(30)
);
```

Explanation: Creates the EMPLOYEE table. Emp_ID uniquely identifies each employee.

Query 2: Create Department Table

SQL

```
CREATE TABLE DEPARTMENT (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(30)
);
```

Explanation: Creates the DEPARTMENT table. Dept_ID uniquely identifies each department.

Query 3: Create Employee-Department Table

SQL

```
CREATE TABLE EMP_DEPT (  
    Emp_ID INT,  
    Dept_ID INT,  
    PRIMARY KEY (Emp_ID, Dept_ID),  
    FOREIGN KEY (Emp_ID) REFERENCES EMPLOYEE(Emp_ID),  
    FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)  
);
```

Explanation: Creates a relationship table between employees and departments. The combination (Emp_ID, Dept_ID) prevents duplicate relationships.

Query 4: Insert Employee Data

SQL

```
INSERT INTO EMPLOYEE VALUES  
(101, 'Arun'),  
(102, 'Priya'),  
(103, 'Ravi');
```

Explanation: Adds employee records to the EMPLOYEE table.

Query 5: Insert Department Data

SQL

```
INSERT INTO DEPARTMENT VALUES  
(10, 'HR'),  
(20, 'Sales'),  
(30, 'IT');
```

Explanation: Adds department records to the DEPARTMENT table.

Query 6: Insert Employee-Department Data

SQL

```
INSERT INTO EMP_DEPT VALUES
```

```
(101, 10),
```

```
(101, 20),
```

```
(103, 20),
```

```
(103, 30);
```

Explanation: Shows which employees belong to which departments. For example, employee **101** works in departments **10** and **20**.

Query 7: Display the Tables

SQL

```
SELECT * FROM EMPLOYEE;
```

```
SELECT * FROM DEPARTMENT;
```

```
SELECT * FROM EMP_DEPT;
```

Explanation: Displays all records from the three tables to verify the BCNF decomposition.

4.A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users


```

mysql> CREATE TABLE Customer (
  -> Customer_ID INT PRIMARY KEY,
  -> Customer_Name VARCHAR(50),
  -> Account_No INT UNIQUE,
  -> Balance DECIMAL(10,2) CHECK (Balance >= 0));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE VIEW Employee_View AS
  -> SELECT Customer_ID, Customer_Name, Account_No
  -> FROM Customer;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE VIEW Manager_View AS
  -> SELECT Customer_ID, Customer_Name, Account_No, Balance
  -> FROM Customer;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Employee_View;
+-----+-----+-----+
| Customer_ID | Customer_Name | Account_No |
+-----+-----+-----+
| 1           | Arun          | 1001       |
| 2           | Priya         | 1002       |
| 3           | Ravi          | 1003       |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Manager_View;
+-----+-----+-----+-----+
| Customer_ID | Customer_Name | Account_No | Balance |
+-----+-----+-----+-----+

```

QUERIES EXPLANATION

The aim is to use views and constraints to provide secure access and maintain correct data.

1. Create the Customer Table

SQL

```

CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(50),
    Account_No INT UNIQUE,
    Balance DECIMAL(10,2) CHECK (Balance >= 0)
);

```

Explanation:

- PRIMARY KEY uniquely identifies each customer.
- UNIQUE prevents duplicate account numbers.

- CHECK (Balance >= 0) prevents negative balances.

2. Create Account Table

SQL

```
CREATE TABLE Account (
    Account_No INT PRIMARY KEY,
    Customer_ID INT,
    Account_Type VARCHAR(20),
    Balance DECIMAL(10,2) CHECK (Balance >= 0),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);
```

Explanation:

- Account_No uniquely identifies an account.
- FOREIGN KEY connects the account to a valid customer.
- CHECK ensures the balance cannot be negative.

3. Create a View for Bank Employees

SQL

```
CREATE VIEW Employee_View AS
SELECT Customer_ID, Customer_Name, Account_No
FROM Customer;
```

Explanation:

The view displays only necessary customer information. Sensitive information such as the balance can be hidden.

4. Create a View for Managers

SQL

```
CREATE VIEW Manager_View AS
SELECT Customer_ID, Customer_Name, Account_No, Balance
```

FROM Customer;

Explanation:

Managers can view customer details along with the account balance.

5. Display the Employee View

SQL

```
SELECT * FROM Employee_View;
```

Explanation:

Displays only the information allowed for bank employees.

6. Display the Manager View

SQL

```
SELECT * FROM Manager_View;
```

Explanation:

Displays customer information including the balance for authorized managers.

Short Conclusion

Views provide controlled access to data, while PRIMARY KEY, UNIQUE, FOREIGN KEY, and CHECK constraints maintain data security and integrity.

5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

```

mysql> CREATE TABLE Customer (
  -> Customer_ID INT PRIMARY KEY,
  -> Customer_Name VARCHAR(50),
  -> Account_No INT UNIQUE,
  -> Balance DECIMAL(10,2) CHECK (Balance >= 0));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE VIEW Employee_View AS
  -> SELECT Customer_ID, Customer_Name, Account_No
  -> FROM Customer;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE VIEW Manager_View AS
  -> SELECT Customer_ID, Customer_Name, Account_No, Balance
  -> FROM Customer;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Employee_View;
+-----+-----+-----+
| Customer_ID | Customer_Name | Account_No |
+-----+-----+-----+
| 1           | Arun          | 1001       |
| 2           | Priya         | 1002       |
| 3           | Ravi          | 1003       |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Manager_View;
+-----+-----+-----+-----+
| Customer_ID | Customer_Name | Account_No | Balance |
+-----+-----+-----+-----+

```

QUERIES EXPLANATION

Scenario: A sales database has redundant data. We normalize it up to **3NF** to remove redundancy and data anomalies.

1. Create the original Sales table

SQL

```

CREATE TABLE Sales (
  Sale_ID INT PRIMARY KEY,
  Customer_ID INT,
  Customer_Name VARCHAR(50),
  Product_ID INT,
  Product_Name VARCHAR(50),

```

Quantity INT

);

Explanation:

This table stores sale, customer, and product details together. This causes **repetition of customer and product information**.

2. Create Customer table

SQL

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(50)  
);
```

Explanation:

Customer details are separated into their own table. This removes repeated customer names.

3. Create Product table

SQL

```
CREATE TABLE Product (  
    Product_ID INT PRIMARY KEY,  
    Product_Name VARCHAR(50)  
);
```

Explanation:

Product details are stored separately, reducing repeated product names.

4. Create Sales table in 3NF

SQL

```
CREATE TABLE Sales (  
    Sale_ID INT PRIMARY KEY,  
    Customer_ID INT,  
    Product_ID INT,
```

```
Quantity INT,  
FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
);
```

Explanation:

The Sales table now contains only information directly related to a sale. Foreign keys connect it to the Customer and Product tables.

5. Insert Customer Data

SQL

```
INSERT INTO Customer VALUES  
(1, 'Arun'),  
(2, 'Priya'),  
(3, 'Ravi');
```

Explanation:

Adds customer records to the Customer table.

6. Insert Product Data

SQL

```
INSERT INTO Product VALUES  
(101, 'Laptop'),  
(102, 'Mouse'),  
(103, 'Keyboard');
```

Explanation:

Adds product records to the Product table.

7. Insert Sales Data

SQL

```
INSERT INTO Sales VALUES  
(1001, 1, 101, 2),  
(1002, 2, 102, 5),
```

(1003, 1, 103, 1);

Explanation:

Stores each sale using Customer_ID and Product_ID instead of repeating their names.

8. Display the normalized data

SQL

SELECT * FROM Customer;

SELECT * FROM Product;

SELECT * FROM Sales;

Explanation:

Displays the three normalized tables and verifies that the data has been separated correctly.

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

1. Display Customer Details

SQL

SELECT * FROM Customer;

Explanation:

Displays all customers from the Customer table.

2. Display Order Details

SQL

SELECT * FROM Orders;

Explanation:

Displays all orders placed by customers.

3. Join Customers and Orders

SQL

SELECT c.Customer_ID, c.Customer_Name, o.Order_ID

FROM Customer c

JOIN Orders o

ON c.Customer_ID = o.Customer_ID;

Explanation:

- JOIN combines the Customer and Orders tables.
- ON c.Customer_ID = o.Customer_ID matches each order with its customer.
- It displays the customer name along with their order ID.

4. Find Frequent Customers

SQL

```
SELECT c.Customer_ID, c.Customer_Name,  
       COUNT(o.Order_ID) AS Order_Count  
FROM Customer c  
JOIN Orders o  
ON c.Customer_ID = o.Customer_ID  
GROUP BY c.Customer_ID, c.Customer_Name  
HAVING COUNT(o.Order_ID) > 2;
```

Explanation:

- COUNT() counts the number of orders made by each customer.
- GROUP BY groups orders for each customer.
- HAVING COUNT(o.Order_ID) > 2 selects customers who placed **more than 2 orders**.

5. Find Customers With Above-Average Orders Using a Subquery

SQL

```
SELECT Customer_ID, Customer_Name  
FROM Customer  
WHERE Customer_ID IN (  
    SELECT Customer_ID  
    FROM Orders  
    GROUP BY Customer_ID  
    HAVING COUNT(o.Order_ID) > (SELECT AVG(Order_Count) FROM Customer_Orders))
```


HAVING COUNT(Order_ID) > 2

);

Explanation:

The **subquery** first finds customers who have more than 2 orders. The outer query then retrieves their names and IDs.

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

```
mysql> SELECT c.Customer_ID, c.Customer_Name,  
->      COUNT(o.Order_ID) AS Order_Count  
-> FROM Customer c  
-> JOIN Orders o  
-> ON c.Customer_ID = o.Customer_ID  
-> GROUP BY c.Customer_ID, c.Customer_Name  
-> HAVING COUNT(o.Order_ID) > 2;
```

```
+-----+-----+-----+  
| Customer_ID | Customer_Name | Order_Count |  
+-----+-----+-----+  
| 1           | Arun           | 3           |  
| 3           | Ravi           | 4           |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```

Explanation

- **Customer_ID** → Unique ID of the customer.

- **Customer_Name** → Name of the customer.
- **Order_Count** → Total number of orders placed by that customer.
- COUNT(Order_ID) counts the orders.
- GROUP BY combines orders belonging to the same customer.
- HAVING COUNT(Order_ID) > 2 displays only customers with **more than 2 orders**.

7. **Scenario:** A relation contains **multi-valued dependencies**. We decompose it into **4NF** to remove redundancy.

```
mysql> SELECT c.Customer_ID, c.Customer_Name,
->      COUNT(o.Order_ID) AS Order_Count
-> FROM Customer c
-> JOIN Orders o
-> ON c.Customer_ID = o.Customer_ID
-> GROUP BY c.Customer_ID, c.Customer_Name
-> HAVING COUNT(o.Order_ID) > 2;
```

Customer_ID	Customer_Name	Order_Count
1	Arun	3
3	Ravi	4

2 rows in set (0.00 sec)

1. Create the Original Table

SQL

```
CREATE TABLE Student_Info (
    Student_ID INT,
    Course VARCHAR(50),
    Hobby VARCHAR(50)
);
```

Explanation:

This table stores a student's courses and hobbies together. If a student has multiple courses and multiple hobbies, many unnecessary combinations are stored.

2. Insert Sample Data

SQL

```
INSERT INTO Student_Info VALUES  
(101, 'DBMS', 'Cricket'),  
(101, 'DBMS', 'Music'),  
(101, 'OS', 'Cricket'),  
(101, 'OS', 'Music');
```

Explanation:

Student 101 has **two courses** and **two hobbies**. This creates repeated combinations.

The multi-valued dependencies are:

Student_ID $\twoheadrightarrow$ Course

Student_ID $\twoheadrightarrow$ Hobby

3. Create Student-Course Table

SQL

```
CREATE TABLE Student_Course (  
    Student_ID INT,  
    Course VARCHAR(50),  
    PRIMARY KEY (Student_ID, Course)  
);
```

Explanation:

Stores only the courses taken by each student.

4. Create Student-Hobby Table

SQL

```
CREATE TABLE Student_Hobby (  
    Student_ID INT,  
    Hobby VARCHAR(50),  
    PRIMARY KEY (Student_ID, Hobby)  
);
```

Explanation:

Stores only the hobbies of each student.

5. Insert Data into Student-Course

SQL

```
INSERT INTO Student_Course VALUES  
(101, 'DBMS'),  
(101, 'OS');
```

Explanation:

Stores each student's courses without repeating hobby information.

6. Insert Data into Student-Hobby

SQL

```
INSERT INTO Student_Hobby VALUES  
(101, 'Cricket'),  
(101, 'Music');
```

Explanation:

Stores each student's hobbies separately without repeating course information.

7. Display the 4NF Tables

SQL

```
SELECT * FROM Student_Course;  
SELECT * FROM Student_Hobby;
```

Explanation:

Displays the decomposed tables and verifies that the multi-valued dependencies have been separated.

8.Scenario: Improper constraints can allow incorrect or inconsistent data. We use **PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK** constraints to maintain data integrity.

```

mysql> CREATE TABLE Department (
  -> Dept_ID INT PRIMARY KEY,
  -> Dept_Name VARCHAR(50) NOT NULL UNIQUE);
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Employee (
  -> Emp_ID INT PRIMARY KEY,
  -> Emp_Name VARCHAR(50) NOT NULL,
  -> Salary DECIMAL(10,2) CHECK (Salary > 0),
  -> Dept_ID INT,
  -> FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID));
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Department VALUES
  -> (10, 'HR'), (20, 'Sales'), (30, 'IT');
Query OK, 3 rows affected (0.01 sec)

mysql> INSERT INTO Employee VALUES
  -> (101, 'Arun', 30000, 10),
  -> (102, 'Priya', 35000, 20),
  -> (103, 'Ravi', 40000, 30);
Query OK, 3 rows affected (0.01 sec)

mysql> INSERT INTO Employee VALUES (104, 'Meena', -5000, 10);
ERROR 3819 (HY000): Check constraint 'employee_chk_1' is violated.

mysql> INSERT INTO Employee VALUES (105, 'Kumar', 25000, 50);
ERROR 1452 (23000): Cannot add or update a child row:
a foreign key constraint fails

mysql> SELECT * FROM Employee;

```

1. Create Department Table

SQL

```

CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50) NOT NULL UNIQUE
);

```

Explanation:

- PRIMARY KEY → uniquely identifies each department.
- NOT NULL → department name cannot be empty.
- UNIQUE → prevents duplicate department names.

2. Create Employee Table

SQL

```

CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,

```

```
Emp_Name VARCHAR(50) NOT NULL,  
Salary DECIMAL(10,2) CHECK (Salary > 0),  
Dept_ID INT,  
FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```

Explanation:

- PRIMARY KEY → uniquely identifies each employee.
- NOT NULL → employee name is required.
- CHECK (Salary > 0) → prevents zero or negative salary.
- FOREIGN KEY → ensures the employee belongs to an existing department.

3. Insert Department Data

SQL

```
INSERT INTO Department VALUES  
(10, 'HR'),  
(20, 'Sales'),  
(30, 'IT');
```

Explanation:

Adds valid department records.

4. Insert Employee Data

SQL

```
INSERT INTO Employee VALUES  
(101, 'Arun', 30000, 10),  
(102, 'Priya', 35000, 20),  
(103, 'Ravi', 40000, 30);
```

Explanation:

Adds employees with valid salary and department values.

5. Test an Invalid Salary

SQL

```
INSERT INTO Employee
```

```
VALUES (104, 'Meena', -5000, 10);
```

Explanation:

This query is rejected because:

```
CHECK (Salary > 0)
```

does not allow a negative salary.

6. Test an Invalid Department

SQL

```
INSERT INTO Employee
```

```
VALUES (105, 'Kumar', 25000, 50);
```

Explanation:

This query is rejected because department 50 does not exist in the Department table.

The **FOREIGN KEY** prevents invalid department values.

9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

```

mysql> CREATE TABLE Student_Project_Rel (
-> Student_ID INT, Project_ID INT,
-> PRIMARY KEY (Student_ID, Project_ID));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student_Course (
-> Student_ID INT, Course_ID INT,
-> PRIMARY KEY (Student_ID, Course_ID));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Project_Course (
-> Project_ID INT, Course_ID INT,
-> PRIMARY KEY (Project_ID, Course_ID));
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student_Project_Rel VALUES
-> (101, 201), (101, 202);
Query OK, 2 rows affected (0.01 sec)

mysql> INSERT INTO Student_Course VALUES
-> (101, 301), (101, 302);
Query OK, 2 rows affected (0.01 sec)

mysql> INSERT INTO Project_Course VALUES
-> (201, 301), (202, 302);
Query OK, 2 rows affected (0.01 sec)

mysql> SELECT * FROM Student_Project_Rel;
+-----+-----+
| Student_ID | Project_ID |

```

QUERIES EXPLANTION

Scenario

A relation contains a **join dependency** that causes redundant data. We decompose it into **5NF**.

Consider:

STUDENT_PROJECT(Student_ID, Project_ID, Course_ID)

A student can work on different projects and take different courses. Instead of storing all combinations in one table, we separate the independent relationships.

1. Create the Original Table

SQL

```

CREATE TABLE Student_Project (
    Student_ID INT,
    Project_ID INT,
    Course_ID INT
);

```

2. Create Student-Project Table

SQL

```
CREATE TABLE Student_Project_Rel (  
    Student_ID INT,  
    Project_ID INT,  
    PRIMARY KEY (Student_ID, Project_ID)  
);
```

3. Create Student-Course Table

SQL

```
CREATE TABLE Student_Course (  
    Student_ID INT,  
    Course_ID INT,  
    PRIMARY KEY (Student_ID, Course_ID)  
);
```

4. Create Project-Course Table

SQL

```
CREATE TABLE Project_Course (  
    Project_ID INT,  
    Course_ID INT,  
    PRIMARY KEY (Project_ID, Course_ID)  
);
```

5. Insert Sample Data

SQL

```
INSERT INTO Student_Project_Rel VALUES  
(101, 201),  
(101, 202);
```

```
INSERT INTO Student_Course VALUES  
(101, 301),  
(101, 302);
```

```
INSERT INTO Project_Course VALUES
```

```
(201, 301),
```

```
(202, 302);
```

6. Display the Decomposed Tables

SQL

```
SELECT * FROM Student_Project_Rel;
```

```
SELECT * FROM Student_Course;
```

```
SELECT * FROM Project_Course;
```

Explanation

- The original table stores three independent relationships together.
- This can create unnecessary combinations and redundancy.
- The relation is decomposed into three smaller tables.
- Each table stores one relationship.
- This decomposition removes redundancy caused by the **join dependency**.

10.Scenario: A system uses inefficient SQL queries. We optimize them using JOINS, indexes, and query restructuring.

1. Create the Tables

SQL

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(50)  
);  
  
CREATE TABLE Orders (  
    Order_ID INT PRIMARY KEY,  
    Customer_ID INT,  
    Order_Date DATE,  
    Amount DECIMAL(10,2),  
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)  
);
```

2. Insert Sample Data

SQL

```
INSERT INTO Customer VALUES  
(1, 'Arun'),  
(2, 'Priya'),  
(3, 'Ravi');
```

```
INSERT INTO Orders VALUES  
(101, 1, '2026-08-01', 5000),  
(102, 2, '2026-08-02', 3000),  
(103, 1, '2026-08-03', 2500);
```

3. Inefficient Query

SQL

```
SELECT *
```

FROM Customer, Orders

WHERE Customer.Customer_ID = Orders.Customer_ID;

Problem:

Using SELECT * retrieves unnecessary columns, and the old comma-style join is less clear.

4. Optimized Query

SQL

SELECT c.Customer_ID, c.Customer_Name,

o.Order_ID, o.Order_Date, o.Amount

FROM Customer c

JOIN Orders o

ON c.Customer_ID = o.Customer_ID;

Explanation:

- JOIN clearly connects the two tables.
- Only required columns are selected.
- This reduces unnecessary data retrieval.

5. Create an Index

SQL

CREATE INDEX idx_customer_id

ON Orders(Customer_ID);

Explanation:

The index helps MySQL find orders for a particular customer faster, especially when the Orders table contains many records.

6. Query Using a Condition

SQL

SELECT c.Customer_Name, o.Order_ID, o.Amount

FROM Customer c

JOIN Orders o

ON c.Customer_ID = o.Customer_ID

WHERE o.Amount > 3000;

Explanation:

The WHERE condition filters the results and returns only orders with an amount greater than **3000**.